Ministerul Educației și Cercetării Al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 3:
Empirical analysis of algorithms: Depth First Search (DFS), Breadth First Search (BFS)

Elaborated:
st. gr. FAF-243 Poiata Calin

Verified:
asist. univ. Fiștic Cristofor

Chișinău – 2026

Table of Contents

ALGORITHM ANALYSIS **3**
Objective 3
Tasks 3
Theoretical Notes 3
Introduction 3
Comparison Metric 4
Input Data 4
IMPLEMENTATION **5**
Breadth First Search (BFS) 5
Depth First Search (DFS) 6
RESULTS **8**
Results Table 8
Results Graph 9
CONCLUSION **10**
ANNEX **11**

ALGORITHM ANALYSIS

Objective

Study and empirically analyze the performance of graph traversal algorithms — Depth First Search (DFS) and Breadth First Search (BFS) — across a variety of graph structures, measuring the time required to locate a target node.

Tasks

1. Implement DFS and BFS graph traversal algorithms in Python;
2. Establish the properties of the input data (graph type, size, target placement) against which the analysis is performed;
3. Choose a metric for comparing the algorithms;
4. Perform empirical analysis of the proposed algorithms across seven distinct graph scenarios;
5. Present the results in a table and graphically using a bar chart;
6. Draw conclusions from the obtained data.

Theoretical Notes

Graph traversal algorithms systematically visit every reachable node from a starting point. Empirical analysis measures real execution time on concrete graph instances, complementing theoretical complexity bounds by revealing the influence of graph structure and target placement.

Standard methodology:

1. Define the analysis goal (time to reach a target node).
2. Select a metric (average wall-clock time in milliseconds).
3. Specify input properties (graph type, number of nodes and edges, start and target nodes).
4. Implement the algorithms.
5. Generate test graphs representative of practical use cases.
6. Execute experiments with multiple repetitions and record results.
7. Analyse, tabulate, and visualise the measurements.

Introduction

Depth First Search and Breadth First Search are the two canonical graph traversal strategies. BFS explores all neighbors of a node before proceeding to their children, advancing level by level. DFS follows each branch as far as possible before backtracking. Both run in $O(V + E)$ time in general, but their practical performance differs significantly depending on graph topology and the relative position of the target node.

- **BFS** uses a FIFO queue whose size can grow to $O(V)$ in wide or dense graphs.
- **DFS** uses a LIFO stack whose depth can reach $O(V)$ in deep or chain-like graphs.

Understanding how these structural differences translate to real execution time for different graph shapes is the subject of this laboratory work.

Comparison Metric

The primary metric is the **average wall-clock execution time** (milliseconds) for the search function to find (or confirm the absence of) a target node. Each scenario is repeated five times and the mean is recorded. The number of expanded nodes and peak frontier size are also tracked internally but the main comparison is based on timing.

Input Data

Seven distinct graph categories are used as inputs:

1. **Chain / Path graph** — a linear sequence of $N = 20,000$ nodes. The target is at the far end (node 19,999). This stresses DFS stack depth and BFS queue width equally, forcing both algorithms to traverse the entire chain.
2. **Balanced Tree** — a tree of depth 9 with branching factor 3 (~ 9,841 nodes). The target is the deepest leaf. This tests the impact of branching on frontier growth and memory.
3. **Dense / Complete graph** — every node connected to every other node ($N = 2,000$, $E = 1,999,000$). The target is the last node. BFS enqueues almost the entire graph in the first step.
4. **Grid graph** — a 120×120 grid (14,400 nodes). The target is the opposite corner. This models realistic 2-D pathfinding, where both algorithms must cover significant area.
5. **Deep vs Shallow target** — the same balanced binary tree (depth 10, 1,023 nodes) is searched twice: once for a shallow target (node 1, direct child of root) and once for the deepest leaf. Isolates the effect of target depth on search time.
6. **Large Random graph** — $N = 4,500$ nodes with edge probability $p = 0.0015$ (sparse, seed-controlled). Tests general scalability on irregular structure.
7. **Disconnected graph** — two separate path components of 4,000 nodes each. The target lies in the second component, which is unreachable from the start node. Tests correct handling of unreachable nodes.

IMPLEMENTATION

Both algorithms are implemented as iterative functions in Python, avoiding recursion depth limits. The benchmarking harness repeats each search five times, records wall-clock nanoseconds via

`time.perf_counter_ns`

, converts to milliseconds, and stores the average.

Breadth First Search (BFS)

Algorithm Description:

BFS starts at the source node and visits all nodes at distance k before visiting nodes at distance $k + 1$. It uses a FIFO queue to manage the frontier and a visited set to avoid revisiting nodes. BFS guarantees that the first time a node is reached it is via a shortest path (in terms of hop count).

Worst-case time complexity: $O(V + E)$. Worst-case space complexity: $O(V)$ for the queue.

Pseudocode:

```
BFS(graph, start, target):
    visited <- {start}
    queue  <- deque([start])
    while queue not empty:
        node <- queue.popleft()
        if node = target: return FOUND
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return NOT FOUND
```

Implementation:

```python
def bfs(graph, start, target):
    visited = {start}
    queue = deque([start])
    peak_frontier = 1
    expanded = 0

    while queue:
        node = queue.popleft()
        expanded += 1

        if node == target:
            return True, expanded, peak_frontier

        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

        if len(queue) > peak_frontier:
```

```
            peak_frontier = len(queue)

    return False, expanded, peak_frontier
```

Depth First Search (DFS)

Algorithm Description:

DFS starts at the source node and explores as far as possible along each branch before backtracking. It uses an explicit LIFO stack instead of recursion, avoiding Python's recursion limit. Neighbors are pushed in reverse order so the leftmost neighbor is explored first, matching the natural recursive order.

Worst-case time complexity: $O(V + E)$. Worst-case space complexity: $O(V)$ for the stack.

Pseudocode:

```
DFS(graph, start, target):
    visited <- {}
    stack   <- [start]
    while stack not empty:
        node <- stack.pop()
        if node in visited: continue
        visited.add(node)
        if node = target: return FOUND
        for neighbor in reversed(graph[node]):
            if neighbor not in visited:
                stack.push(neighbor)
    return NOT FOUND
```

Implementation:

```
def dfs(graph, start, target):
    visited = set()
    stack = [start]
    peak_frontier = 1
    expanded = 0

    while stack:
        node = stack.pop()
        if node in visited:
            continue

        visited.add(node)
        expanded += 1

        if node == target:
            return True, expanded, peak_frontier

        neighbors = graph.get(node, [])
        for neighbor in reversed(neighbors):
            if neighbor not in visited:
                stack.append(neighbor)

        if len(stack) > peak_frontier:
```

```
        peak_frontier = len(stack)

    return False, expanded, peak_frontier
```

RESULTS

The program was executed on all seven graph scenarios. Each search was repeated five times and the average time (ms) was recorded. The table below summarises the findings.

Results Table

Scenario	Algorithm	Nodes	Edges	Target Type	Found	Avg Time (ms)
Chain/Path	BFS	20000	19999	deep target	True	10.5057
Chain/Path	DFS	20000	19999	deep target	True	12.7548
Balanced Tree	BFS	9841	9840	deep leaf	True	4.2275
Balanced Tree	DFS	9841	9840	deep leaf	True	5.3738
Dense/Complete	BFS	2000	1999000	any neighbor	True	197.9138
Dense/Complete	DFS	2000	1999000	any neighbor	True	187.0823
Grid	BFS	14400	28560	opposite corner	True	8.6990
Grid	DFS	14400	28560	opposite corner	True	10.8632
Deep vs Shallow	BFS	1023	1022	shallow	True	0.0035
Deep vs Shallow	DFS	1023	1022	shallow	True	0.0029
Deep vs Shallow	BFS	1023	1022	deep	True	0.4456
Deep vs Shallow	DFS	1023	1022	deep	True	0.5786
Large Random	BFS	4500	15367	random target	True	2.5300
Large Random	DFS	4500	15367	random target	True	2.9881
Disconnected	BFS	8000	7998	unreachable	False	1.8360
Disconnected	DFS	8000	7998	unreachable	False	2.2776

Figure 1 BFS vs DFS benchmark results across all graph scenarios

Key observations from the table:

- In **6 of 8** scenario-target comparisons, BFS is faster than DFS; only the **Deep vs Shallow (shallow)** and **Dense / Complete** cases favour DFS.
- On **Chain / Path**, BFS is faster (10.5057 ms) than DFS (12.7548 ms), despite both traversing the same 20,000-node linear structure.
- On **Dense / Complete**, DFS is faster (187.0823 ms vs 197.9138 ms), consistent with BFS paying higher frontier-management cost on very dense connectivity.
- On **Deep vs Shallow**, both are near-instant for shallow target (0.0035 ms BFS, 0.0029 ms DFS), while BFS is faster for deep target (0.4456 ms vs 0.5786 ms).
- On **Disconnected**, both correctly return

  `False`

  , and BFS remains faster (1.8360 ms vs 2.2776 ms).

Results Graph

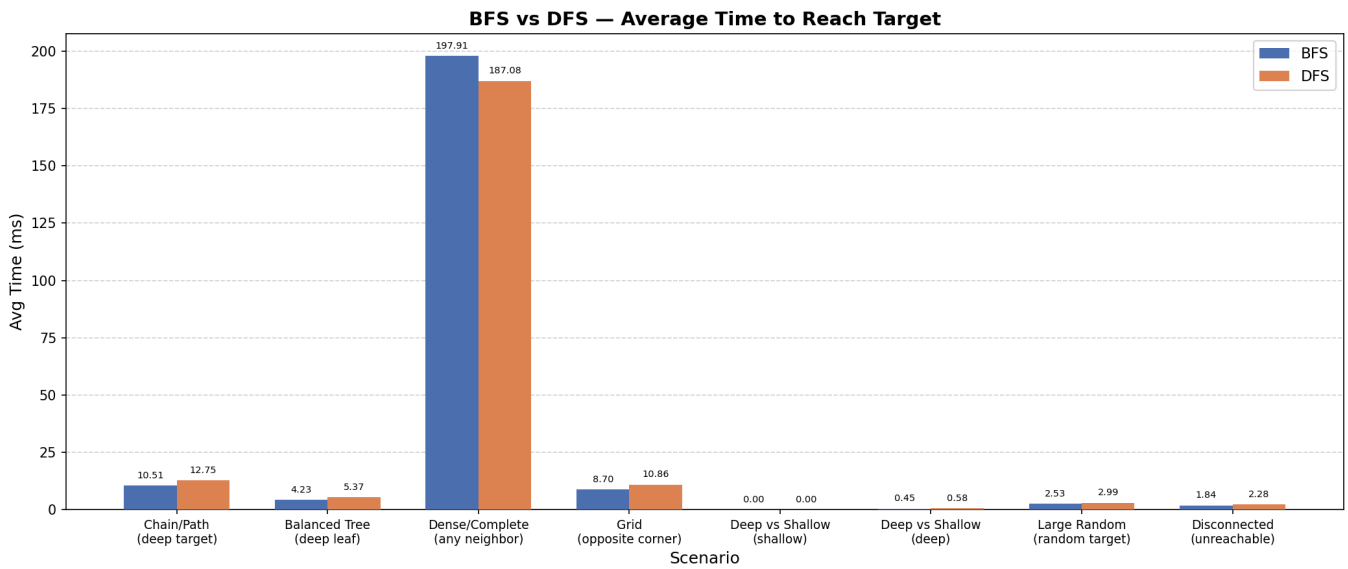

Figure 2 BFS vs DFS --- average time to reach target (bar chart per scenario)

The bar chart reinforces the table data visually. Notable points:

- The **Dense / Complete** scenario clearly dominates runtime for both methods, and is the only large case where DFS outperforms BFS by a noticeable margin.
- For **Balanced Tree**, **Grid**, and **Large Random**, BFS bars are consistently lower than DFS, indicating better practical performance on these datasets.
- The **Deep vs Shallow (shallow)** bars are effectively near zero for both algorithms, showing immediate target discovery.
- The **Disconnected** case confirms correct failure handling (

  `Found = False`

) with BFS still requiring less time than DFS.

CONCLUSION

The empirical results confirm that while DFS and BFS share the same asymptotic complexity $O(V + E)$, their practical performance differs markedly depending on graph structure and target placement.

BFS is the stronger overall performer in this experiment, delivering lower average time in most tested scenarios (chain/path, balanced tree, grid, deep target, large random, disconnected). It remains especially suitable when shortest-path guarantees are required.

DFS still has competitive behavior and wins in specific cases, particularly the dense/complete graph and the shallow-target microcase, where stack-based traversal overhead is lower than BFS frontier expansion cost.

For **grid / pathfinding** scenarios both algorithms perform similarly, though BFS would be preferred in practice because it guarantees the shortest path.

For graphs with **disconnected components**, both correctly terminate with "not found" after exhausting the reachable subgraph — no special handling is required beyond the standard visited set.

In conclusion, algorithm choice should be guided by graph structure and query type: use BFS as the default for robust practical performance and shortest-path correctness, while DFS remains a good alternative for specific dense or extremely shallow-target cases.

ANNEX

Github Repository